

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Ananya Galagali (1BM24CS040)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Ananya Galagali (1BM24CS040)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (23CS4PCADA) work prescribed for the said degree.

Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	10
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	13
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	16
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	22
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	26
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	30 33
9	Implement Fractional Knapsack using Greedy technique.	36
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	39
11	Implement "N-Queens Problem" using Backtracking.	42

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Topological Sorting

```
#include <stdio.h>

#define MAX 10

int main() {
    int adj[MAX][MAX], indegree[MAX];
    int queue[MAX], front = 0, rear = 0;
    int n, i, j, count = 0, v;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
    for(i = 0; i < n; i++) {
        indegree[i] = 0;
    }
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(adj[i][j] == 1)
                indegree[j]++;
        }
    }
    for(i = 0; i < n; i++) {
        if(indegree[i] == 0)
            queue[rear++] = i;
    }
    printf("Topological Order: ");
```

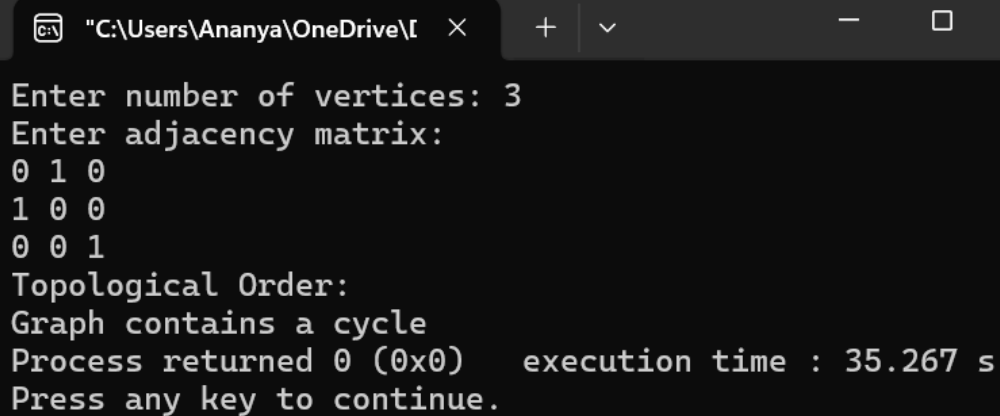
```

while(front < rear) {
    v = queue[front++];
    printf("%d ", v);
    count++;
    for(j = 0; j < n; j++) {
        if(adj[v][j] == 1) {
            indegree[j]--;
            if(indegree[j] == 0)
                queue[rear++] = j;
        }
    }
}
if(count != n)
    printf("\nGraph contains a cycle");
return 0;
}

```

OUTPUT:

Default: "C:\Users\Ananya\OneDrive\Documents
 \Ananya 040\Topological sort.exe"
ctrl+alt+1



```

C:\Users\Ananya\OneDrive\I x + v - □ ×
Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
1 0 0
0 0 1
Topological Order:
Graph contains a cycle
Process returned 0 (0x0)    execution time : 35.267 s
Press any key to continue.

```

Leetcode Problem (207.Course Schedule):

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
```

```
    int* indegree = (int*)calloc(numCourses, sizeof(int));
```

```
    int** graph = (int**)malloc(numCourses * sizeof(int*));
```

```
    int* graphColSize = (int*)calloc(numCourses, sizeof(int));
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        graph[i] = (int*)malloc(numCourses * sizeof(int)); // worst case
```

```
    }
```

```
    for (int i = 0; i < prerequisitesSize; i++) {
```

```
        int course = prerequisites[i][0];
```

```
        int prereq = prerequisites[i][1];
```

```
        graph[prereq][graphColSize[prereq]++] = course;
```

```
        indegree[course]++;
```

```
    }
```

```
    int* queue = (int*)malloc(numCourses * sizeof(int));
```

```
    int front = 0, rear = 0;
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        if (indegree[i] == 0) {
```

```
            queue[rear++] = i;
```

```
        }
```

```
    }
```

```
    int count = 0;
```

```
    while (front < rear) {
```

```
        int current = queue[front++];
```

```
        count++;
```

```
        for (int i = 0; i < graphColSize[current]; i++) {
```

```
            int neighbor = graph[current][i];
```

```
            indegree[neighbor]--;
```

```
        if (indegree[neighbor] == 0) {  
            queue[rear++] = neighbor;  
        }  
    }  
}  
for (int i = 0; i < numCourses; i++) {  
    free(graph[i]);  
}  
free(graph);  
free(graphColSize);  
free(indegree);  
free(queue);  
return count == numCourses;  
}
```

OUTPUT:

☒ Testcase | **>_ Test Result** ⌵

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

☒ Testcase | **>_ Test Result** ⌵

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

numCourses =
2

prerequisites =
[[1,0],[0,1]]

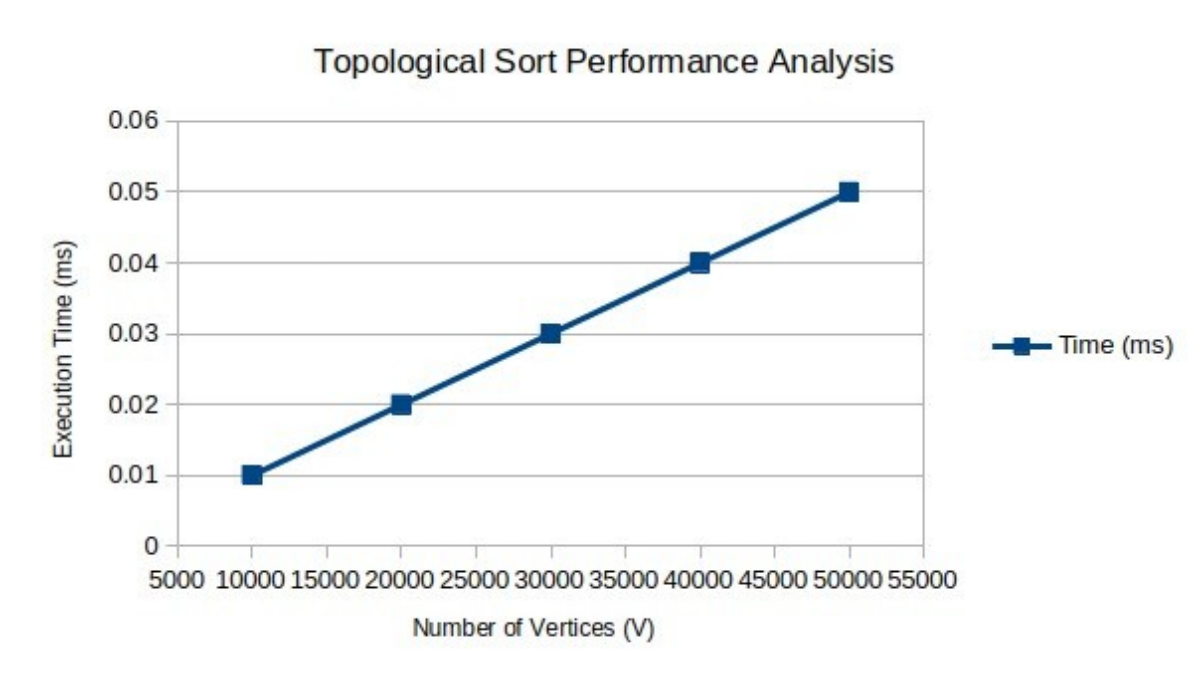
Output

false

Expected

false

GRAPH:



Lab program 2: Johnson Trotter Algorithm

```
#include <stdio.h>
#define SIZE 5
int a[SIZE], max, pos, n;
char dir[SIZE];
void print()
{
    for (int i=0; i<n; i++)
    {
        printf("%d(%c) ", a[i], dir[i]);
    }
    printf("\n");
}
int main()
{
    printf("Enter n value: ");
    scanf("%d", &n);
    for (int i=0; i<n; i++)
    {
        a[i] = i+1;
        dir[i] = 'L';
    }
    print();
    while (1)
    {
        max = -1;
        pos = -1;
        for (int i=0; i<n; i++)
        {
            if (((dir[i] == 'L' && i>0 && a[i] > a[i-1]) ||
                (dir[i] == 'R' && i<n-1 && a[i] > a[i+1])))
            {
                if (a[i] > max)
                {
                    max = a[i];
                }
            }
        }
    }
}
```

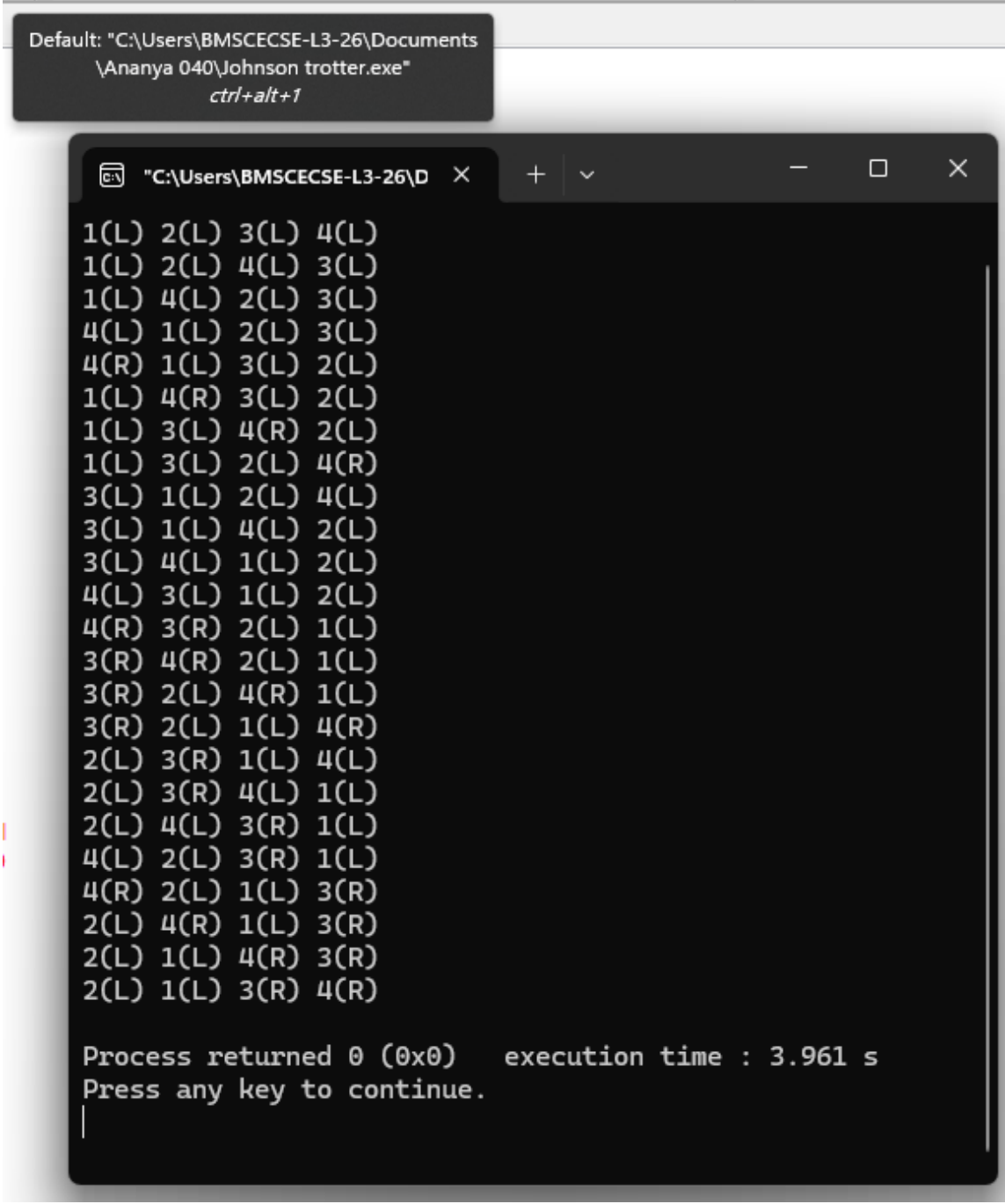
```

        pos = i;
    }
}
}
if (max == -1)
    break;
if (dir[pos] == 'L')
{
    int temp = a[pos];
    a[pos] = a[pos-1];
    a[pos-1] = temp;
    char tempdir = dir[pos];
    dir[pos] = dir[pos-1];
    dir[pos-1] = tempdir;
}
else
{
    int temp = a[pos];
    a[pos] = a[pos+1];
    a[pos+1] = temp;
    char tempdir = dir[pos];
    dir[pos] = dir[pos+1];
    dir[pos+1] = tempdir;
}
for (int i=0; i<n; i++)
{
    if (a[i] > max)
    {
        char d = dir[i];
        if (d == 'L')
            dir[i] = 'R';
        else
            dir[i] = 'L';
    }
}
}

```

```
    print();  
}  
return 0;  
}
```

OUTPUT:



```
Default: "C:\Users\BMSCECSE-L3-26\Documents  
  \Ananya 040\Johnson trotter.exe"  
      ctrl+alt+1
```

```
"C:\Users\BMSCECSE-L3-26\Documents\Ananya 040\Johnson trotter.exe" X + - □ X  
1(L) 2(L) 3(L) 4(L)  
1(L) 2(L) 4(L) 3(L)  
1(L) 4(L) 2(L) 3(L)  
4(L) 1(L) 2(L) 3(L)  
4(R) 1(L) 3(L) 2(L)  
1(L) 4(R) 3(L) 2(L)  
1(L) 3(L) 4(R) 2(L)  
1(L) 3(L) 2(L) 4(R)  
3(L) 1(L) 2(L) 4(L)  
3(L) 1(L) 4(L) 2(L)  
3(L) 4(L) 1(L) 2(L)  
4(L) 3(L) 1(L) 2(L)  
4(R) 3(R) 2(L) 1(L)  
3(R) 4(R) 2(L) 1(L)  
3(R) 2(L) 4(R) 1(L)  
3(R) 2(L) 1(L) 4(R)  
2(L) 3(R) 1(L) 4(L)  
2(L) 3(R) 4(L) 1(L)  
2(L) 4(L) 3(R) 1(L)  
4(L) 2(L) 3(R) 1(L)  
4(R) 2(L) 1(L) 3(R)  
2(L) 4(R) 1(L) 3(R)  
2(L) 1(L) 4(R) 3(R)  
2(L) 1(L) 3(R) 4(R)  
  
Process returned 0 (0x0)   execution time : 3.961 s  
Press any key to continue.  
|
```

Lab program 3: Merge Sort

```
#include <stdio.h>
#include<time.h>
int a[20],n;
void simple_sort(int [],int,int,int);
void merge_sort(int[],int,int);
int main()
{
    int i;
    clock_t start, end;
    double time_taken;
    printf("Enter the no. of elements:");
    scanf("%d", &n);
    printf("Enter the array elements:");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    merge_sort(a, 0, n - 1);
    printf("Sorted array:");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
void merge_sort(int a[],int low, int high){
    if(low<high){
        int mid=(low+high)/2;
        merge_sort(a,low,mid);
        merge_sort(a,mid+1,high);
        simple_sort(a,low,mid,high);
    }
}
void simple_sort(int a[],int low, int mid, int high){
    int i=low,j=mid+1,k=low;
    int c[n];
```

```

while(i<=mid && j<=high){
    if(a[i]<a[j]){
        c[k++]=a[i];
        i++;
    }else{
        c[k++]=a[j];
        j++;
    }
}
while(i<=mid){
    c[k++]=a[i];
    i++;
}
while(j<=high){
    c[k++]=a[j];
    j++;
}
for(i=low;i<=high;i++){
    a[i]=c[i];
}
}

```

OUTPUT:

```

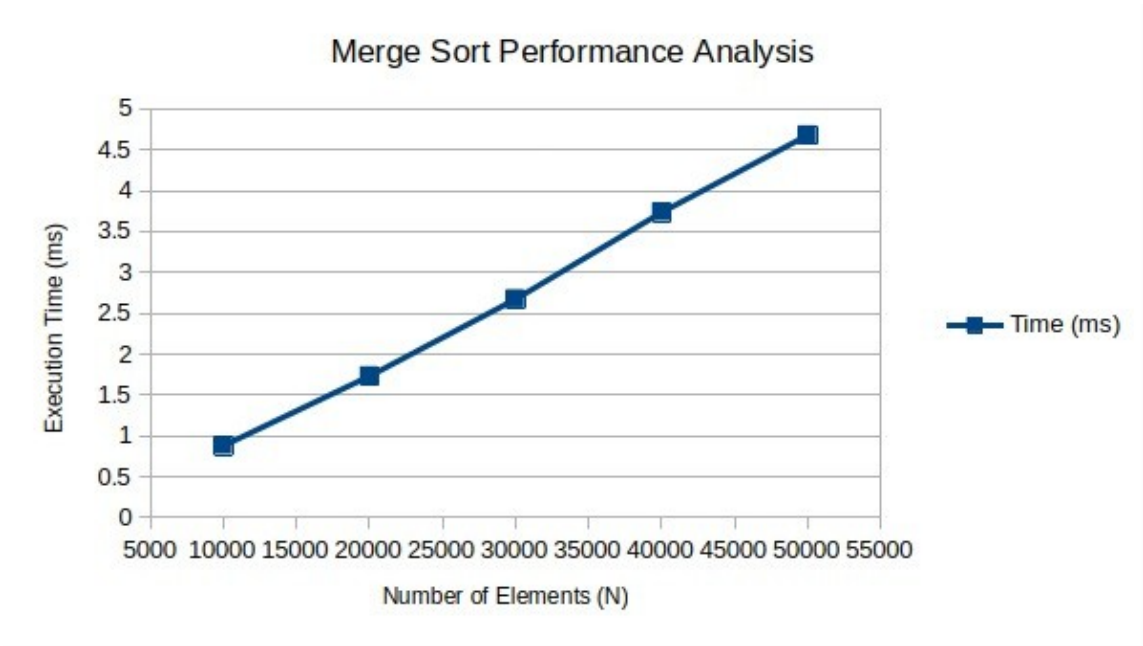
Default: "C:\Users\BMSCECSE-L3-26\Documents
\Ananya 040\Maerge sort.exe"
ctrl+alt+1

"C:\Users\BMSCECSE-L3-26\Documents\Ananya 040\Maerge sort.exe"
Enter the no. of elements:8
Enter the array elements:3 14 56 89 12 19 65 83
Sorted array:3 12 14 19 56 65 83 89
Time taken to sort: 0.000000 seconds

Process returned 0 (0x0)   execution time : 23.804 s
Press any key to continue.
|

```

GRAPH:



Lab program 4: Quick Sort

```
#include <stdio.h>
#include<time.h>
int a[20],n;
int partition(int [],int, int);
void quick_sort(int [],int,int);
void swap(int*,int*);
int main()
{
    int i;
    clock_t start, end;
    double time_taken;
    printf("Enter the no. of elements:");
    scanf("%d", &n);
    printf("Enter the array elements:");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    start = clock();
    quick_sort(a, 0, n - 1);
    end = clock();
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Sorted array:");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    printf("Time taken to sort: %f seconds\n", time_taken);
    return 0;
}

void swap(int *a,int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}

void quick_sort(int a[],int low,int high){
```

```

    if(low<high){
        int mid=partition(a,low,high);
        quick_sort(a,low,mid-1);
        quick_sort(a,mid+1,high);
    }
}

int partition(int a[],int low,int high){
    int pivot=a[low];
    int i=low;
    int j=high+1;
    while(i<=j){
        do{
            i=i+1;
        }while(a[i]<pivot && i<=high);
        do{
            j=j-1;
        }while(a[j]>pivot && j>=low);
        if(i<j){
            swap(&a[i],&a[j]);
        }
    }
    swap(&a[j],&a[low]);
    return j;
}

```

OUTPUT:

```

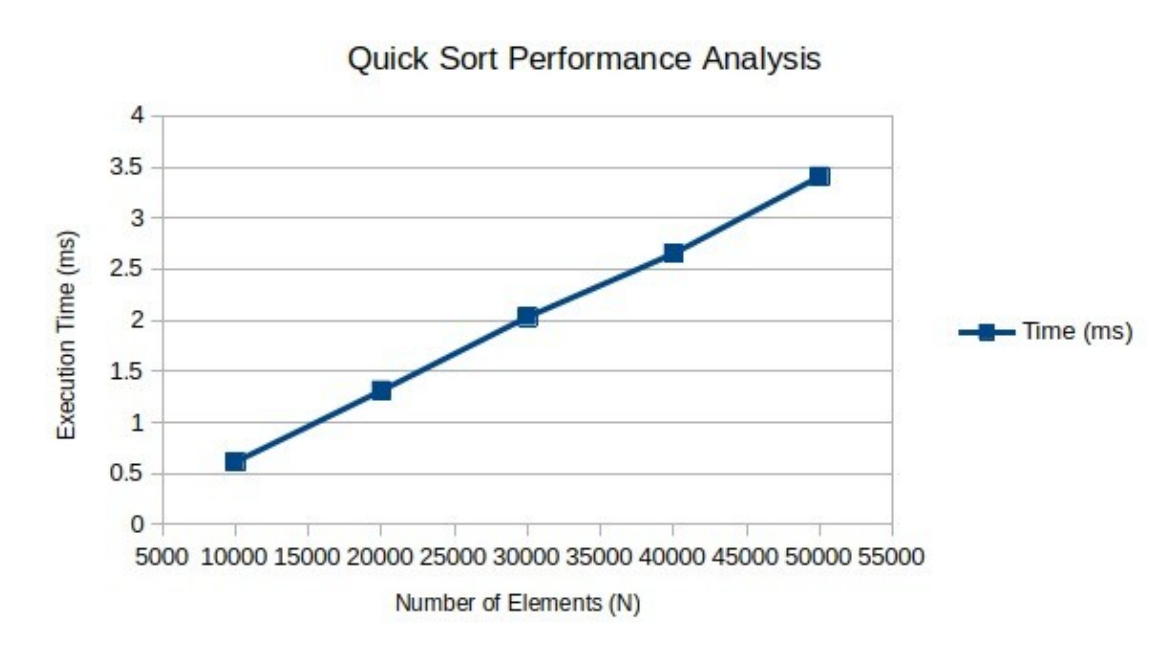
Default: "C:\Users\BMSCECSE-L3-26\Documents
\Ananya 040\QS.exe"
ctrl+alt+1

C:\Users\BMSCECSE-L3-26\Documents\Ananya 040\QS.exe
Enter the no. of elements:8
Enter the array elements:36 8 59 2 40 28 64 17
Sorted array:2 8 17 28 36 40 59 64
Time taken to sort: 0.000000 seconds

Process returned 0 (0x0)   execution time : 42.276 s
Press any key to continue.
|

```

GRAPH:



Lab program 5: Heap Sort

```
#include <stdio.h>
#include <time.h>
void heapify(int a[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;
    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i)
    {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;
        heapify(a, n, largest);
    }
}
void heapSort(int a[], int n)
{
    int i, temp;
    for (i = n / 2 - 1; i >= 0; i--)
    {
        heapify(a, n, i);
    }
    for (i = n - 1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a, i, 0);
    }
}
```

```
int main()
{
    int a[1000], n, i;
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    start = clock();
    heapSort(a, n);
    end = clock();
    printf("\nSorted array:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\n\nTime taken = %f seconds\n", time_taken);
    return 0;
}
```

OUTPUT:

Default: "C:\Users\Ananya\OneDrive\Documents
\Ananya 040\Heap sort.exe"
ctrl+alt+1

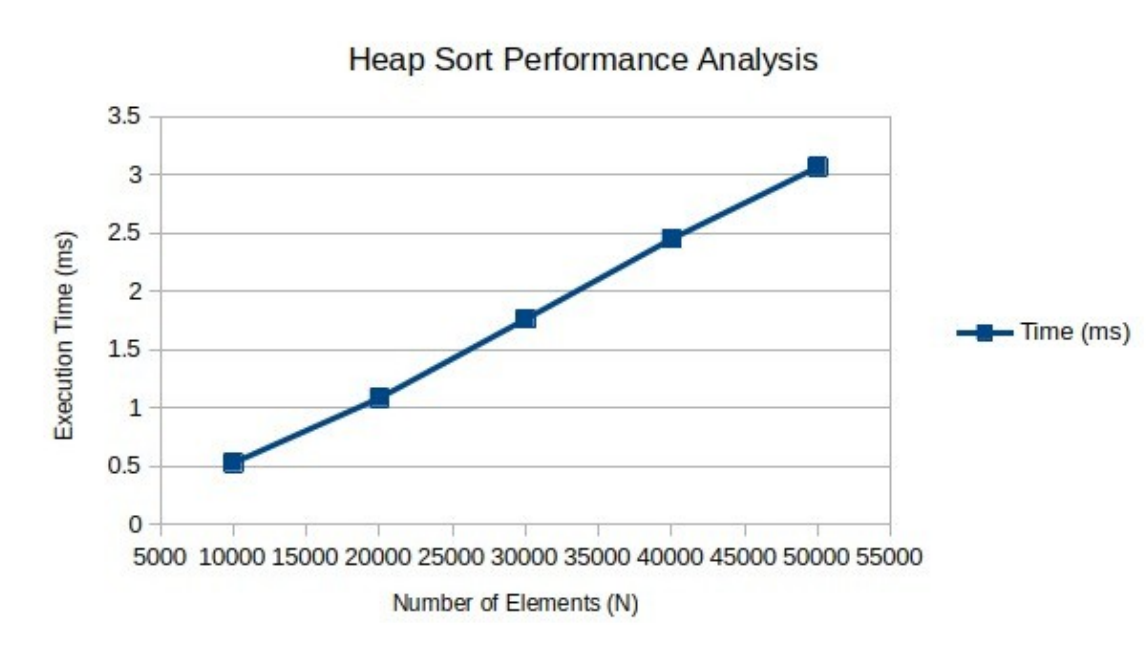
```
"C:\Users\Ananya\OneDrive\I  x  +  v  -  □  x
Enter number of elements: 6
Enter elements:
70 89 65 43 36 45

Sorted array:
36 43 45 65 70 89

Time taken = 0.000000 seconds

Process returned 0 (0x0)  execution time : 47.826 s
Press any key to continue.
```

GRAPH:



Lab program 6: 0/1 Knapsack Problem Using Dynamic Programming

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int n, int M, int W[], int P[]) {
    int i, j;
    int table[n + 1][M + 1];
    for (i = 0; i <= n; i++)
        table[i][0] = 0;
    for (j = 0; j <= M; j++)
        table[0][j] = 0;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= M; j++) {
            if (j < W[i - 1]) {
                table[i][j] = table[i - 1][j];
            } else {
                table[i][j] = max(
                    table[i - 1][j],
                    P[i - 1] + table[i - 1][j - W[i - 1]]
                );
            }
        }
    }
    return table[n][M];
}

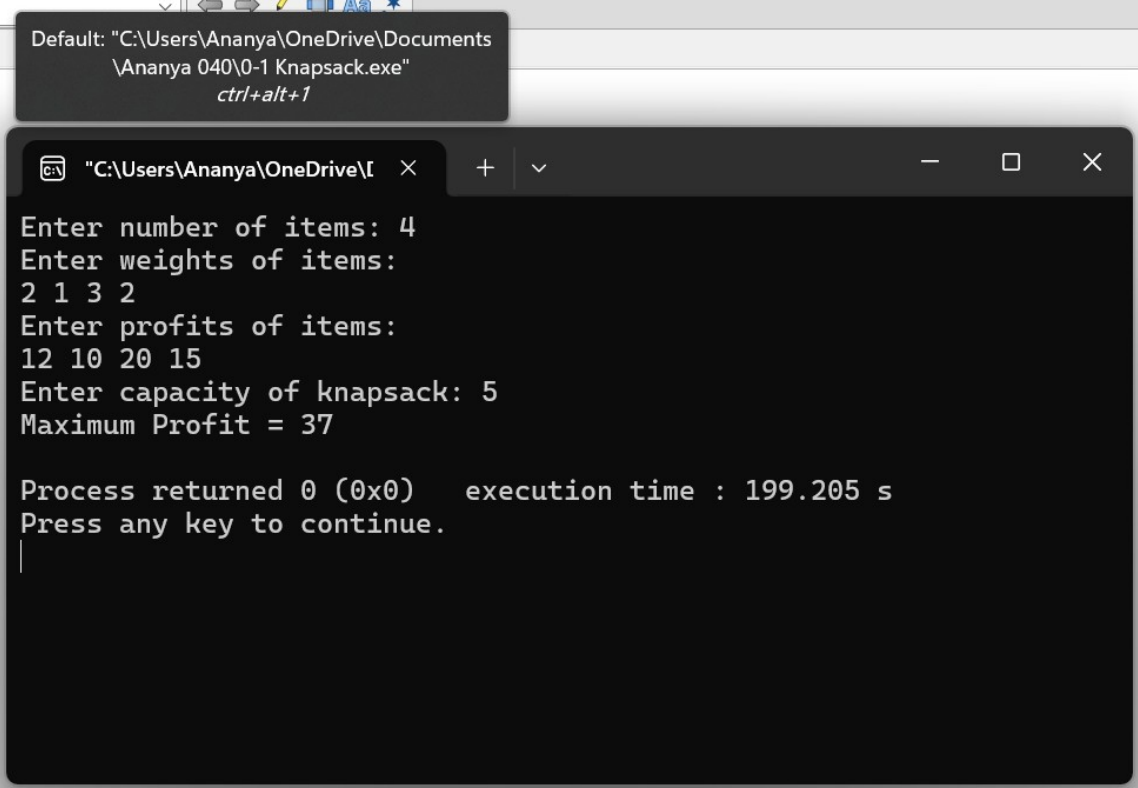
int main() {
    int n, M;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int W[n], P[n];
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &W[i]);
    }
}
```

```

printf("Enter profits of items:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &P[i]);
}
printf("Enter capacity of knapsack: ");
scanf("%d", &M);
int result = knapsack(n, M, W, P);
printf("Maximum Profit = %d\n", result);
return 0;
}

```

OUTPUT:



```

Default: "C:\Users\Ananya\OneDrive\Documents
\Ananya 040\0-1 Knapsack.exe"
ctrl+alt+1

"C:\Users\Ananya\OneDrive\Documents\Ananya 040\0-1 Knapsack.exe"
Enter number of items: 4
Enter weights of items:
2 1 3 2
Enter profits of items:
12 10 20 15
Enter capacity of knapsack: 5
Maximum Profit = 37

Process returned 0 (0x0)   execution time : 199.205 s
Press any key to continue.

```

Leetcode Problem (801.Minimum Swaps):

```
#include <stdio.h>

#include <limits.h>

int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {

    int keep = 0; // No swap at current position
    int swap = 1; // Swap at current position
    for (int i = 1; i < nums1Size; i++) {

        int newKeep = INT_MAX;
        int newSwap = INT_MAX;
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
            newKeep = keep;
            newSwap = swap + 1;
        }
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
            if (swap < newKeep)
                newKeep = swap;
            if (keep + 1 < newSwap)
                newSwap = keep + 1;
        }
        keep = newKeep;
        swap = newSwap;
    }
    return (keep < swap) ? keep : swap;
}
```

OUTPUT:

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums1 =
[1,3,5,4]

nums2 =
[1,2,3,7]

Output

1

Expected

1

☒ Testcase | [Test Result](#) ⌵

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums1 =
[0,3,5,8,9]

nums2 =
[2,1,4,6,9]

Output

1

Expected

1

Lab program 7: Floyd's Algorithm

```
#include <stdio.h>

int a[10][10],D[10][10],n;

void floyd(int a[][10],int);

int min(int,int);

int main()
{
    printf("Enter the no. of vertices:");
    scanf("%d",&n);
    printf("Enter the cost adjacency matrix:\n");
    int i,j;
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&a[i][j]);
        }
    }
    floyd(a,n);
    printf("Distance Matrix:\n");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            printf("%d ",D[i][j]);
        }
        printf("\n");
    }
    return 0;
}

void floyd(int a[][10],int n){
    int i,j,k;
    for(i=0;i<n;i++){
```

```

        for(j=0;j<n;j++){
            D[i][j]=a[i][j];
        }
    }
    for(k=0;k<n;k++){
        for(i=0;i<n;i++){
            for(j=0;j<n;j++){
                D[i][j]=min(D[i][j],(D[i][k]+D[k][j]));
            }
        }
    }
}

int min(int a,int b){
    if(a<b){
        return a;
    }else{
        return b;
    }
}

```

OUTPUT:

```

Default: "C:\Users\Ananya\OneDrive\Documents
\Ananya 040\Floyds.exe"
ctrl+alt+1

"C:\Users\Ananya\OneDrive\Documents\Ananya 040\Floyds.exe"
Enter the no. of vertices:4
Enter the cost adjacency matrix:
0 5 99 10
99 0 3 99
99 99 0 1
99 99 99 0
Distance Matrix:
0 5 8 9
99 0 3 4
99 99 0 1
99 99 99 0

Process returned 0 (0x0)   execution time : 43.749 s
Press any key to continue.

```

Leetcode Problem (287.Duplicate Number):

```
int findDuplicate(int* nums, int numsSize)
{
    int slow = nums[0];
    int fast = nums[0];
    while (1)
    {
        slow = nums[slow];
        fast = nums[nums[fast]];
        if (slow == fast)
            break;
    }
    slow = nums[0];
    while (slow != fast)
    {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

OUTPUT:

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[1,3,4,2,2]

Output

2

Expected

2

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[3,1,3,4,2]

Output

3

Expected

3

Lab program 8:(a) Prim's Algorithm

```
#include <stdio.h>

#define INF 9999

#define MAX 10

void prims(int n, int cost[MAX][MAX]) {

    int near[MAX], t[MAX][2];

    int i, j, k, l, min, mincost = 0;

    min = INF;

    for (i = 1; i <= n; i++) {

        for (j = i + 1; j <= n; j++) {

            if (cost[i][j] < min) {

                min = cost[i][j];

                k = i;

                l = j;

            }

        }

        t[1][1] = k;

        t[1][2] = l;

        mincost = cost[k][l];

        for (i = 1; i <= n; i++) {

            if (cost[i][k] < cost[i][l])

                near[i] = k;

            else

                near[i] = l;

        }

        near[k] = 0;

    }
```

```

near[1] = 0;
for (i = 2; i <= n - 1; i++) {
    min = INF;
    for (j = 1; j <= n; j++) {
        if (near[j] != 0 && cost[j][near[j]] < min) {
            min = cost[j][near[j]];
            k = j;
        }
    }
    t[i][1] = k;
    t[i][2] = near[k];
    mincost += cost[k][near[k]];
    near[k] = 0;
    for (j = 1; j <= n; j++) {
        if (near[j] != 0 && cost[j][near[j]] > cost[j][k]) {
            near[j] = k;
        }
    }
}

printf("\nEdges in MST:\n");
for (i = 1; i <= n - 1; i++) {
    printf("%d - %d\n", t[i][1], t[i][2]);
}

printf("Minimum cost = %d\n",
mincost);
}

int main() {

```

```

int n, cost[MAX][MAX];

int i, j;

printf("Enter number of vertices: ");

scanf("%d", &n);

printf("Enter cost matrix:\n");

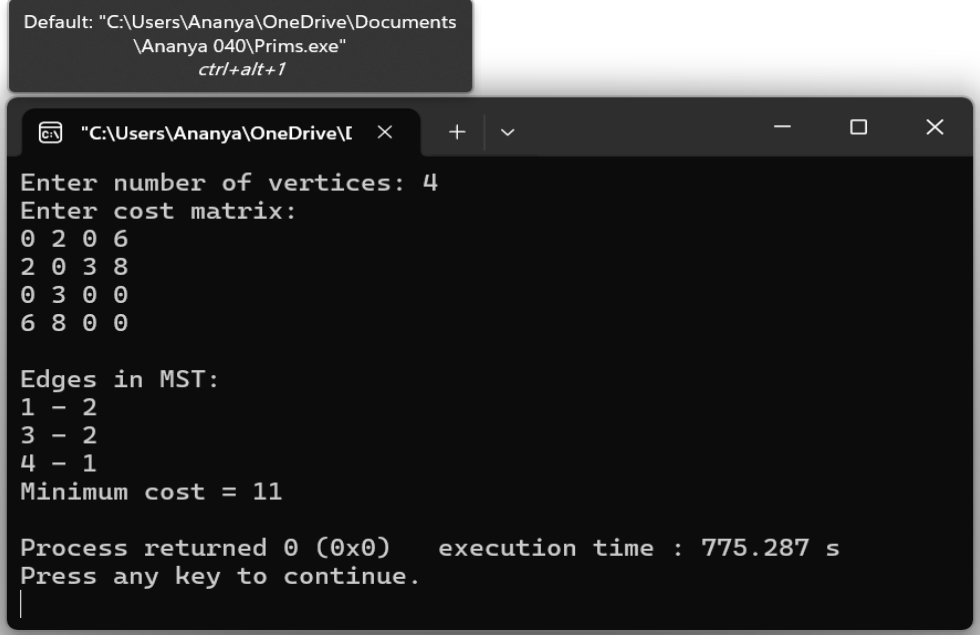
for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j++) {
        scanf("%d", &cost[i][j]);
        if (cost[i][j] == 0)
            cost[i][j] = INF;
    }
}

prims(n, cost);

return 0;
}

```

OUTPUT:



```

Default: "C:\Users\Ananya\OneDrive\Documents
\Ananya 040\Prims.exe"
ctrl+alt+1

"C:\Users\Ananya\OneDrive\I  x + v - □ x
Enter number of vertices: 4
Enter cost matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0

Edges in MST:
1 - 2
3 - 2
4 - 1
Minimum cost = 11

Process returned 0 (0x0) execution time : 775.287 s
Press any key to continue.

```

Lab program 8:(b) Kruskal's Algorithm

```

#include <stdio.h>

#define MAX 10

#define INF 9999

int parent[MAX];

int find(int i) {
    while (parent[i])
        i = parent[i];
    return i;
}

void union_set(int i, int j) {
    parent[j] = i;
}

int main() {
    int n, cost[MAX][MAX];
    int t[MAX][2];
    int i, j, u, v, k;
    int min, mincost = 0;
    int edge_count = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

```

```

    }

    for (i = 1; i <= n; i++)
        parent[i] = 0;

    while (edge_count < n - 1) {
        min = INF;
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= n; j++) {
                if (cost[i][j] < min) {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }

        i = find(u);
        j = find(v);

        if (i != j) {
            edge_count++;
            t[edge_count][1] = u;
            t[edge_count][2] = v;
            mincost += min;
            union_set(i, j);
        }

        cost[u][v] = cost[v][u] = INF;
    }

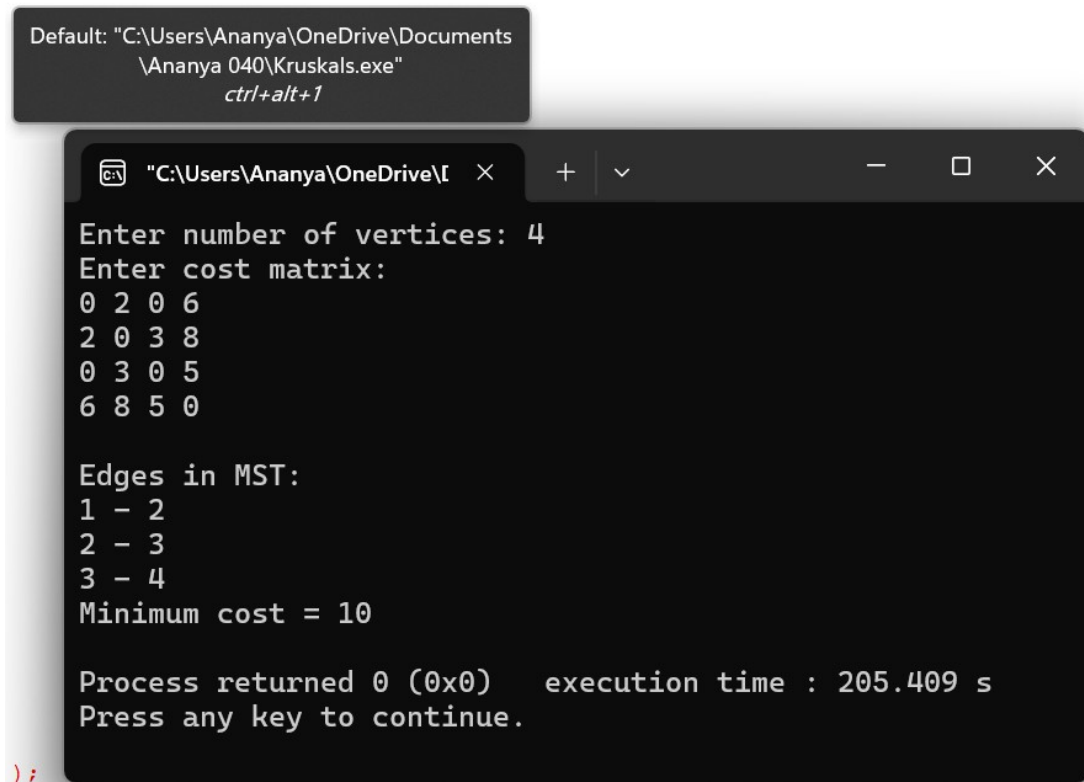
    printf("\nEdges in MST:\n");

    for (i = 1; i <= edge_count; i++) {

```

```
        printf("%d - %d\n", t[i][1], t[i][2]);  
    }  
  
    printf("Minimum cost = %d\n",  
mincost);  
  
    return 0;  
}
```

OUTPUT:



```
Default: "C:\Users\Ananya\OneDrive\Documents  
Ananya 040\Kruskals.exe"  
ctrl+alt+1  
  
"C:\Users\Ananya\OneDrive\Documents\Ananya 040\Kruskals.exe"  
Enter number of vertices: 4  
Enter cost matrix:  
0 2 0 6  
2 0 3 8  
0 3 0 5  
6 8 5 0  
  
Edges in MST:  
1 - 2  
2 - 3  
3 - 4  
Minimum cost = 10  
  
Process returned 0 (0x0)   execution time : 205.409 s  
Press any key to continue.  
) ;
```

Lab program 9: Fractional Knapsack Using Greedy Technique

```
#include<stdio.h>

int main()
{
    float p[100],w[100],x[100],m,total;

    int n,k;

    printf("enter the number of items");

    scanf("%d",&n);

    printf("enter the capacity of
knapsack:");

    scanf("%f",&m);

    for(int i=0;i<n;i++)
    {
        printf("enter profit value %d: ",i+1);

        scanf("%f",&p[i]);

        x[i]=0;

        printf("enter weight value %d: ",i+1);

        scanf("%f",&w[i]);
    }

    int temp;

    for(int i=0;i<n-1;i++)
    {
        for(int j=i+1;j<n;j++)
        {
            if(p[i]/w[i]<p[j]/w[j])
            {
                temp=p[i];
```

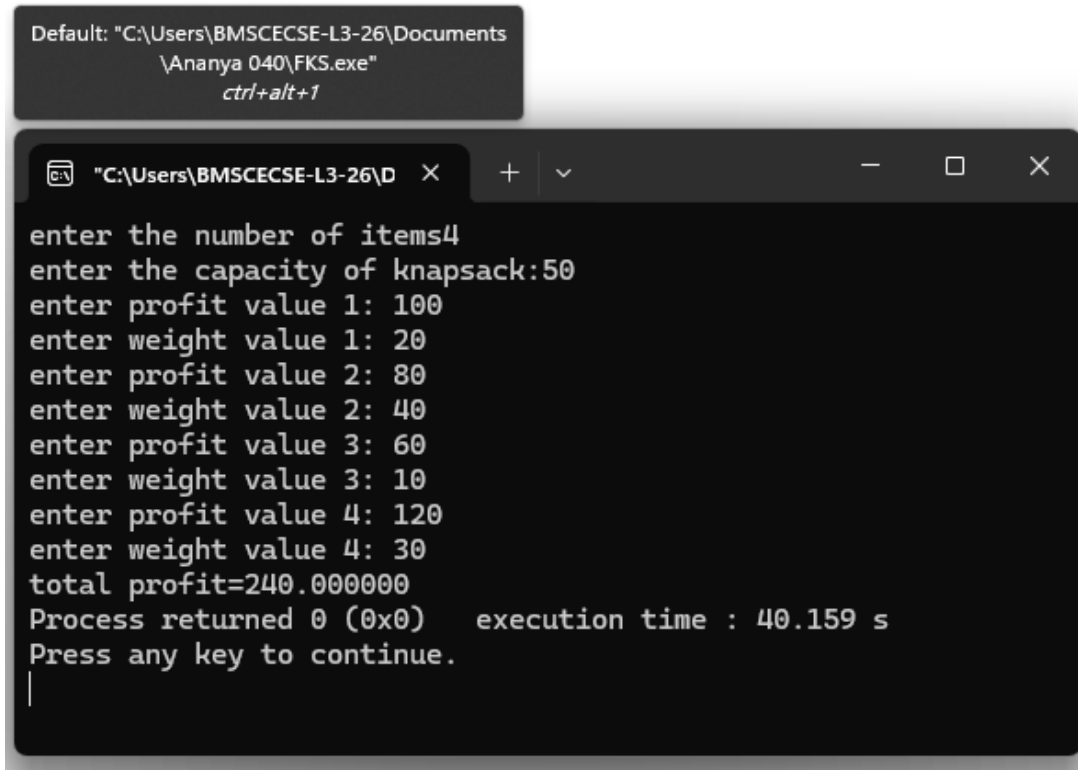
```

        p[i]=p[j];
        p[j]=temp;
        temp=w[i];
        w[i]=w[j];
        w[j]=temp;
    }
}
}
int rem_cap=m;
for(k=0;k<n;k++)
{
    if(w[k]>rem_cap)
    {
        break;
    }
    rem_cap=rem_cap-w[k];
    x[k]=1;
}
if(k<n)
{
    x[k]=rem_cap/w[k];
}
for(int i=0;i<n;i++)
{
    total=total+x[i]*p[i];
}

```

```
printf("total profit=%f",total);  
  
return 0;  
  
}
```

OUTPUT:



```
Default: "C:\Users\BMSCECSE-L3-26\Documents  
Ananya 040\FKS.exe"  
ctrl+alt+1  
"C:\Users\BMSCECSE-L3-26\Documents\Ananya 040\FKS.exe" X + - □ X  
enter the number of items4  
enter the capacity of knapsack:50  
enter profit value 1: 100  
enter weight value 1: 20  
enter profit value 2: 80  
enter weight value 2: 40  
enter profit value 3: 60  
enter weight value 3: 10  
enter profit value 4: 120  
enter weight value 4: 30  
total profit=240.000000  
Process returned 0 (0x0) execution time : 40.159 s  
Press any key to continue.  
|
```

Lab program 10: Dijkstra's Algorithm

```
#include <stdio.h>

#define INF 99999

int minDistance(int dist[], int visited[], int
n) {

    int min = INF, min_index = -1;

    for (int v = 0; v < n; v++) {

        if (!visited[v] && dist[v] <= min) {

            min = dist[v];

            min_index = v;

        }

    }

    return min_index;
}

void dijkstra(int n, int graph[n][n], int src)
{

    int dist[n], visited[n];

    for (int i = 0; i < n; i++) {

        dist[i] = INF;

        visited[i] = 0;

    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {

        int u = minDistance(dist, visited, n);

        if (u == -1) break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
```

```

        if (!visited[v] && graph[u][v] &&
            dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u]
                [v];
            }
        }
    }

    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
}

int main() {
    int n;

    printf("Enter the number of vertices of the graph:\n");
    scanf("%d", &n);

    int graph[n][n];

    printf("Enter the adjacency matrix values:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    dijkstra(n, graph, 0);

    return 0;
}

```

OUTPUT:

Default: "C:\Users\BMSCECSE-L3-26\Documents
\Ananya 040\djk.exe"
ctrl+alt+1

```
"C:\Users\BMSCECSE-L3-26\Documents" x + v - □ x
Enter the number of vertices of the graph:
5
Enter the adjacency matrix values:
9999 2 4 9999 9999
2 9999 1 7 9999
4 1 9999 3 5
9999 7 3 9999 1
9999 9999 5 1 9999
Vertex Distance from Source
0 0
1 2
2 3
3 6
4 7

Process returned 0 (0x0) execution time : 54.260 s
Press any key to continue.
|
```

Lab program 11:N-Queens Problem Using Backtracking

```
#include <stdio.h>

#include <stdlib.h>

int x[20];

int count = 0;

int place(int k, int i)
{
    int j;

    for (j = 1; j < k; j++)
    {
        if (x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }

    return 1;
}

void nQueens(int k, int n)
{
    int i, j;

    for (i = 1; i <= n; i++)
    {
        if (place(k, i))
        {
            x[k] = i;
```

```

        if (k == n)
        {
            count++;

            printf("\nSolution %d:\n", count);

            for (j = 1; j <= n; j++)
            {
                for (i = 1; i <= n; i++)
                {
                    if (x[j] == i)
                        printf("Q ");
                    else
                        printf(". ");
                }

                printf("\n");
            }
        }
        else
        {
            nQueens(k + 1, n);
        }
    }
}

```

```
int main()
```

```

{
    int n;

    printf("Enter number of queens: ");
    scanf("%d", &n);

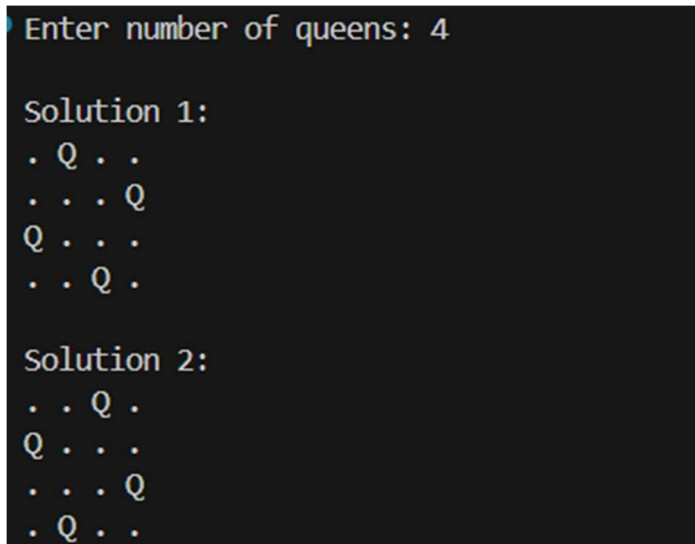
    nQueens(1, n);

    if (count == 0)
        printf("No solution exists");

    return 0;
}

```

OUTPUT:



```

Enter number of queens: 4

Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

```